Strategic Roadmap: Task 3 (Build)

A 150-Minute Sprint to a Production-Ready Minimal Ledger

Step 1
Data Model & Schema

Step 2
Interface & Scaffolding

Step 3
Core Logic & Atomicity

Step 4
Idempotency

Step 5
Polish & Documentation

Step 6
MVP Trade-Offs

Milestone 1: Data Model & Schema

Define the foundation and constraints.

30 Minutes

Specific

Design schema for Accounts, Ledger_Entries, and Idempotency_Keys using strict numeric types (or integer cents).

Achievable

Utilize standard PostgreSQL relational database principles with CHECK constraints.

Measurable

Deliver exact SQL table definitions that enforce double-entry at the database constraint level.

Relevant (Rubric)

Directly fulfills the 'Technical Depth' and 'Double-entry model' rubric requirements.

Codebase / Submission Strategy

`File 1: schema.sql`

Start here. Write pure SQL. Define `accounts` (id, owner). Define `entries` (id, account_id, amount, type: DEBIT/CREDIT, transaction_id). Define `idempotency_keys` (key, status, response). Crucial: Add a constraint so `amount` cannot be negative, forcing correct DEBIT/CREDIT usage.

Milestone 2: Interface & Scaffolding

Establish the service contract.

20 Minutes

Specific

Define empty methods for CreateAccount, PostTransaction, and FetchBalance with strict payload typing.

Achievable

Standard structural setup in your chosen language (Go/Python).

Measurable

A compiling/running file containing the interface structure.

Relevant (Rubric)

Ensures adherence to the 'keep it simple but correct' constraint.

Codebase / Submission Strategy

`File 2: types.go (or models.py)`

Create the data structures (e.g., structs/dataclasses) that map to your SQL tables. Define the `LedgerService` interface. By defining the interface first, you decouple the business logic from the database implementation, scoring high on 'System Design Thinking'.

Milestone 3: Core Logic & Atomicity

Implement the financial transaction engine.

45 Minutes

Specific

Write the logic to execute a debit and credit within a single DB BEGIN/COMMIT transaction block.

Achievable

Focus on the 'happy path' transfer first, then wrap in a database transaction.

Measurable

Code successfully processes a transfer and cleanly rolls back on an error.

Relevant (Rubric)

Proves 'Working Code' and understanding of financial system atomicity.

Codebase / Submission Strategy

`File 3: repository.go (or repository.py)`

Implement the interface. Write the `PostTransaction` method. Code evolution: 1) Open DB Tx. 2) Insert Debit entry for Source account. 3) Insert Credit entry for Destination account (using same transaction_id). 4) Commit. If step 2 or 3 fails, Rollback. Add logic to handle 'deposits' (system-to-user) vs 'transfers' (user-to-user).

Milestone 4: Idempotency

Protect the ledger from duplicate requests.

30 Minutes

Specific

Implement a check at the start of PostTransaction to verify the incoming idempotency key against the DB.

Achievable

Standard caching and database lookup pattern.

Measurable

Logic explicitly blocks a transaction from being recorded twice, returning the cached response.

Relevant (Rubric)

Directly answers the 'Idempotency' requirement and feeds into Task 4 (Reliability).

Codebase / Submission Strategy

`Refactoring: repository.go / service.go`

Evolve `PostTransaction` to accept an `IdempotencyKey` string. Before opening the ledger DB transaction, attempt to INSERT the key into `idempotency_keys`. Use a unique constraint violation (e.g., ON CONFLICT) to detect duplicates. If duplicate, abort ledger update and return the cached status.

Milestone 5: Polish & Documentation

Format for leadership presentation.

25 Minutes

Specific

Review code, add architectural 'why' comments, and format as a clean appendix block.

Achievable

Requires only code review and formatting skills.

Measurable

A fully commented, readable code snippet ready to be pasted into the final submission.

Relevant (Rubric)

Hits the 'Code Quality' and 'Leadership Communication' pillars of the rubric.

Codebase / Submission Strategy

`File 4: main.go (or run.py) + Formatting`

Write a tiny script that instantiates your LedgerService and runs a mock Deposit and Transfer. Paste the core schema, interface, and repository implementation into your presentation appendix. Ensure comments explain *why* you chose dynamic balance calculation over a static column.

Milestone 6: MVP Trade-Offs

Strategic scope limitations to protect the 60-day launch.

Strategic

Currency Precision at the Edge

The ledger stores BIGINT minor units. Currency-to-exponent mapping (e.g., USD=2, JPY=0) is intentionally pushed to the API gateway. This keeps the core ledger strictly mathematical and simple for the initial launch.

Transport-Layer UUID Validation

Repository queries cast strings directly to UUIDs. Strict regex/UUID validation is deferred to the HTTP/Transport layer. This protects database performance and separates domain logic from edge validation.

CQRS Balance Architecture (Fast-Follow)

The MVP uses a dynamic SUM() for absolute mathematical correctness. At high scale, this will be transitioned to a CQRS read-optimized cache updated asynchronously via Kafka or similar event streams.

Codebase / Submission Strategy

MVP Scope & Execution Repository

These documented trade-offs demonstrate a pragmatic approach to the strict 60-day launch window. By intentionally constraining the initial scope, we guarantee the delivery of a mathematically flawless core ledger while establishing a clear fast-follow roadmap for high-scale architectural improvements.

View MVP Repository on GitHub

https://github.com/hallilibas-if/MalCode